

INT396 · UNSUPERVISED LEARNING

## UNIT 3

# Hierarchical and Density-Based Clustering

Two families that do not need  $k$  in advance - one builds a tree of every possible grouping, the other grows clusters through dense regions.

Agglomerative vs Divisive · Single · Complete · Average · Ward · Dendrograms · DBSCAN ·  $\epsilon$ -neighbourhood · MinPts · Noise & Border Points · k-Means vs Hierarchical vs DBSCAN

## UNIT III ROADMAP

# What This Unit Covers

## UNIT 3

*Neither family asks you for  $k$  in advance - which is their main advantage over Unit II.*

- Hierarchical clustering
  - Agglomerative (bottom-up) and divisive (top-down)
  - Four linkage methods: single, complete, average, Ward
  - Reading and cutting a dendrogram
- Density-based clustering
  - DBSCAN fundamentals
  - The  $\epsilon$ -neighbourhood and MinPts parameters
  - Core, border and noise points
- Comparison
  - k-Means vs Hierarchical vs DBSCAN
  - Which assumption each one makes, and where each fails

**Hierarchical**

build the whole tree, cut later

**Linkage**

how you measure cluster-to-cluster

**DBSCAN**

grow through dense regions

**Comparison**

three algorithms, three assumptions

## 01

## SECTION

# Hierarchical Clustering

## IN THIS SECTION

- Agglomerative: start with  $n$  clusters, merge until one remains
- Divisive: start with one cluster, split until  $n$  remain
- Why the whole hierarchy is more useful than one partition
- The cost you pay for it

### 3.1 AGGLOMERATIVE VS DIVISIVE

# Agglomerative vs Divisive

## UNIT 3

*Same hierarchy, built from opposite ends. One of them is practical.*

|                  | Agglomerative (bottom-up)                                            | Divisive (top-down)                                              |
|------------------|----------------------------------------------------------------------|------------------------------------------------------------------|
| Start from       | n clusters - every point alone                                       | One cluster containing everything                                |
| Each step        | Merge the two closest clusters                                       | Split one cluster into two                                       |
| Stop when        | A single cluster remains                                             | Every point is alone                                             |
| Decisions made   | $n - 1$ merges                                                       | $n - 1$ splits                                                   |
| Cost             | $O(n^2 \log n)$ with a heap; $O(n^2)$ memory for the distance matrix | $O(2^n)$ if done exhaustively - needs heuristics to be practical |
| Used in practice | Almost always - this is what scipy and sklearn implement             | Rarely - the search space at each split is enormous              |

**REASON IT OUT** Why is splitting so much more expensive than merging?

### 3.1 HIERARCHICAL CLUSTERING

#### UNIT 3

# Why Build the Whole Hierarchy?

**KEY CONCEPT** · You choose the number of clusters **AFTER** the algorithm runs

**k-Means makes you commit first** you supply  $k$  before it runs, and changing your mind means running it again.

**Hierarchical computes every grouping once** from  $n$  clusters down to one, in a single pass.

**You then cut the tree** at whatever height gives the number of clusters you want - no re-run required.

**Which matters when** you do not know how many groups exist, which in unsupervised work is most of the time.

**It is also deterministic** no random initialisation, so the same data gives the same answer every time - unlike k-Means.

**The price**  $O(n^2)$  memory for the distance matrix, so it does not scale to millions of rows.

# 02

## SECTION

# Linkage Methods

### IN THIS SECTION

- Single linkage - the two closest members
- Complete linkage - the two furthest members
- Average linkage - the mean over all pairs
- Ward - minimise the increase in within-cluster variance

## 3.2 LINKAGE METHODS

## UNIT 3

# Four Ways to Measure the Distance Between Two Clusters

*The points do not change. Only how you measure cluster-to-cluster distance changes - and that changes the tree.*

**SINGLE LINKAGE (NEAREST NEIGHBOUR)**

$$D(A, B) = \min \{ d(a, b) : a \in A, b \in B \}$$

The two closest members. Produces long, chain-like clusters - good for elongated shapes, prone to chaining.

**COMPLETE LINKAGE (FURTHEST NEIGHBOUR)**

$$D(A, B) = \max \{ d(a, b) : a \in A, b \in B \}$$

The two furthest members. Produces compact, roughly equal-sized clusters. Sensitive to outliers.

**AVERAGE LINKAGE**

$$D(A, B) = (1 / |A||B|) \sum_{a \in A} \sum_{b \in B} d(a, b)$$

Mean distance over all cross-pairs. A compromise between single and complete.

**WARD LINKAGE**

$$D(A, B) = \text{increase in total within-cluster sum of squares after merging}$$

Merges the pair that increases WCSS least. Closest in spirit to k-Means, and the usual default.

**CHOOSING BETWEEN THEM**

**Single** follows curves and chains; use for elongated structure

**Complete** compact balls; use when clusters should be similar in size

**Average** a reasonable middle ground

**Ward** the sensible default for numeric data; requires Euclidean distance

**No best** each suits a different shape of data

### 3.2 LINKAGE METHODS

## The Chaining Effect in Single Linkage

### UNIT 3

#### COMMON PITFALL · Its greatest weakness and its greatest strength

**What happens** single linkage merges on the distance between the two CLOSEST members, so a thin thread of intermediate points welds two otherwise separate groups together.

**The symptom** one enormous cluster plus several tiny ones, rather than a balanced partition.

**Why it is a weakness** on blob-shaped data a few noise points bridging two clusters will merge them.

**Why it is a strength** on genuinely elongated shapes - two crescents, a spiral, a river network - chaining is exactly the behaviour you want.

**Complete linkage does the opposite** it uses the two FURTHEST members, so a cluster only forms when every pair is close. Compact, but it will split an elongated cluster in two.

**The lesson** there is no best linkage. There is a linkage that matches the shape of your data.

## 03

## SECTION

# Reading a Dendrogram

## IN THIS SECTION

- What the axes mean
- Height equals dissimilarity
- Cutting the tree to obtain k clusters
- What a dendrogram tells you that a label list cannot

### 3.3 DENDROGRAM INTERPRETATION

# How to Read a Dendrogram

## UNIT 3

*Height means dissimilarity. Everything else follows from that.*

#### THE LEAVES

##### Individual data points

- Along the bottom axis
- Every point starts as its own cluster
- Their left-right order is cosmetic, not meaningful

#### THE VERTICAL AXIS

##### Merge distance

- The height at which two clusters joined
- Low join = the two are very alike
- High join = they are very different

#### A HORIZONTAL CUT

##### Choosing k

- Slice across at any height
- Count the vertical lines you cross
- That count is your number of clusters

#### A LONG VERTICAL GAP

##### Natural structure

- A large jump before the next merge
- Suggests the clusters below are genuinely separate
- Cutting inside that gap is defensible

**TRY IT** If you slide the cut line UP, does k go up or down?

### 3.3 WORKED EXAMPLE

## Cutting a Dendrogram

### UNIT 3

#### THE PROBLEM

- Six items: A, B, C, D, E, F
- Merge heights: A-B at 1.1, E-F at 1.2, C-D at 1.5
- Then (AB)-(CD) at 3.0, and finally ((AB)(CD))-(EF) at 4.6
- Task: how many clusters does a cut at height 2.3 give?

#### WHY IT MATTERS

- Count the vertical lines the cut line crosses
- A tall gap with no merges is the natural place to cut
- Nothing is recomputed - the hierarchy was built once

#### STEP-BY-STEP SOLUTION

**Step 1 - list the merges below the cut** A-B at 1.1 ✓, E-F at 1.2 ✓, C-D at 1.5 ✓ - all below 2.3

**Step 2 - list the merges above the cut** (AB)-(CD) at 3.0 ✗, ((AB)(CD))-(EF) at 4.6 ✗ - both above

**Step 3 - the surviving groups** {A, B}, {C, D}, {E, F}

**Answer** three clusters

**Now cut at 3.5 instead** the merge at 3.0 is now below the cut, so {A,B} and {C,D} join: {A, B, C, D} and {E, F} - two clusters

**The point** one tree, many answers. You chose k AFTER the algorithm finished.

**ANSWER** Cut at 2.3 → 3 clusters. Cut at 3.5 → 2 clusters. Same tree.

# 04

## SECTION

# DBSCAN

### IN THIS SECTION

- Density-Based Spatial Clustering of Applications with Noise
- The two parameters:  $\epsilon$  and MinPts
- Core, border and noise points
- Clusters of any shape - and the assumption that limits it

### 3.4 DBSCAN FUNDAMENTALS

#### UNIT 3

## The Two Definitions DBSCAN Is Built On

*DBSCAN asks 'is this region dense?', not 'which centre is nearest?'*

#### THE E-NEIGHBOURHOOD OF A POINT

$$N_{\epsilon}(p) = \{ q \in D : d(p, q) \leq \epsilon \}$$

Every point within distance  $\epsilon$  of  $p$ .  $\epsilon$  sets the radius of the neighbourhood you look in.

#### CORE POINT CONDITION

$$p \text{ is a CORE point} \Leftrightarrow |N_{\epsilon}(p)| \geq \text{MinPts}$$

A point is core when its neighbourhood contains at least MinPts points, including itself. Core points are the interior of a cluster.

#### THE THREE KINDS OF POINT

**CORE · BORDER · NOISE**

BORDER: within  $\epsilon$  of a core point, but not core itself. NOISE: neither - it belongs to no cluster at all.

#### THE PARAMETERS

$\epsilon$  (epsilon) the neighbourhood radius  
- how close counts as close

**MinPts** how many neighbours make a region dense

**Rule of thumb**  $\text{MinPts} \approx 2 \times \text{number of features}$ ; then read  $\epsilon$  off the knee of a k-distance plot

**No k** the number of clusters is discovered, not supplied

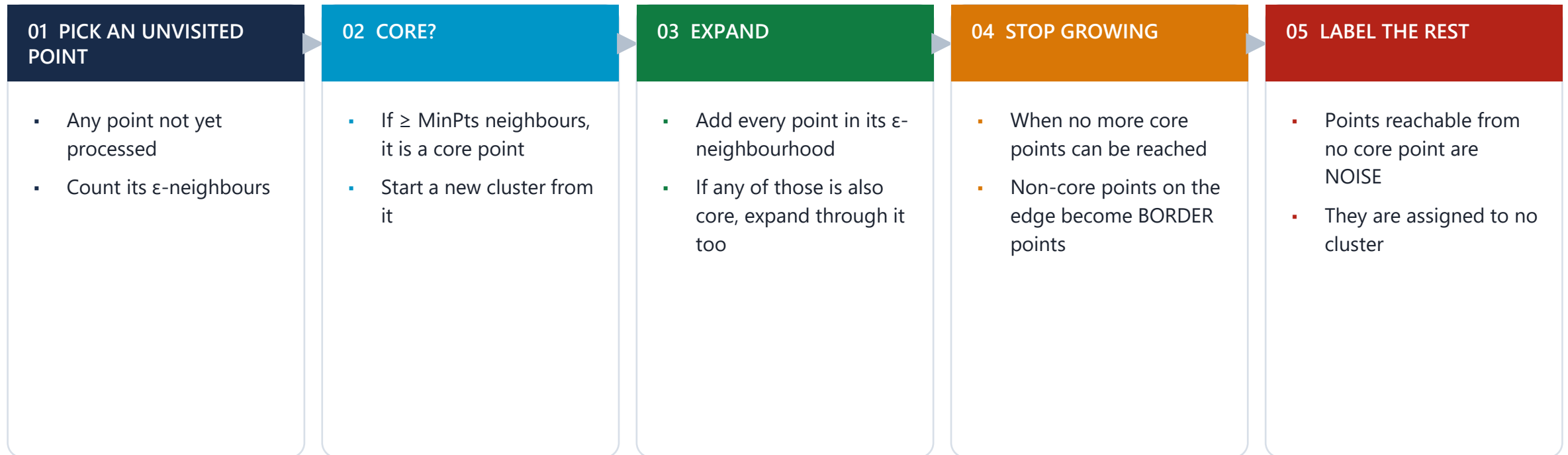
**Noise label** sklearn returns -1 for noise points

### 3.4 DBSCAN

## UNIT 3

# How DBSCAN Builds a Cluster

*Because clusters grow outwards through density, they can follow any shape.*



**CONSIDER** Why can DBSCAN find a crescent when k-Means cannot?

### 3.4 DBSCAN

## UNIT 3

# What DBSCAN Gains, and What It Assumes

## KEY CONCEPT · Strengths and the one assumption that limits it

**Finds arbitrary shapes** crescents, rings and spirals are all fine, because clusters grow through density rather than radiating from a centre.

**Discovers k** the number of clusters comes out of the algorithm rather than going into it.

**Labels noise explicitly** the only algorithm in this course allowed to say a point belongs nowhere - which makes it natural for anomaly detection.

**Deterministic for core points** no random initialisation, so results are reproducible.

**THE LIMITATION - uniform density**  $\epsilon$  and MinPts are global. One tight cluster beside one loose cluster cannot both be captured: a small  $\epsilon$  calls the loose one noise, a large  $\epsilon$  merges the tight one away.

**Also** it degrades in high dimensions, because distances concentrate and 'dense' loses meaning.

## When density varies

- HDBSCAN and OPTICS extend DBSCAN to varying density
- Both are outside this syllabus, but worth knowing the names exist

## CHECK YOUR UNDERSTANDING

# Quick Check - Sections 1 to 4

## UNIT 3

**QUICK CHECK**

You run DBSCAN with  $\text{eps} = 0.2$  and get 90% of points labelled as noise. What is the most likely cause?

- A** The data genuinely has no structure
- B**  $\text{eps}$  is too small, so almost no point reaches MinPts neighbours
- C** MinPts is too small, so no cluster can form
- D** DBSCAN cannot handle more than two features

**ANSWER: B** A small  $\text{eps}$  means each neighbourhood contains very few points, so almost nothing satisfies the core-point condition and everything is left as noise. Read  $\text{eps}$  off the knee of a k-distance plot rather than guessing. A too-small MinPts would do the opposite - it would make clusters form too easily.

## 05

## SECTION

# Comparing the Three Families

## IN THIS SECTION

- k-Means, hierarchical and DBSCAN side by side
- The assumption each one makes
- Which data shape suits which algorithm
- How to choose when you have no labels

### 3.5 COMPARISON

## k-Means vs Hierarchical vs DBSCAN

### UNIT 3

*No algorithm wins everywhere. Each fails exactly where its assumption breaks.*

|                    | k-Means                                          | Hierarchical                        | DBSCAN                   |
|--------------------|--------------------------------------------------|-------------------------------------|--------------------------|
| Need k in advance? | Yes                                              | No - cut the tree afterwards        | No - discovered          |
| Cluster shape      | Convex blobs only                                | Depends on linkage                  | Any shape                |
| Handles noise?     | No - every point is assigned                     | No - every point is assigned        | Yes - labels it -1       |
| Deterministic?     | No - depends on initialisation                   | Yes                                 | Yes for core points      |
| Main assumption    | Spherical, similar-sized clusters                | A meaningful merge criterion exists | Uniform density          |
| Scales to large n? | Yes, especially MiniBatch                        | No - $O(n^2)$ memory                | Moderately               |
| Fails when         | Clusters are elongated or of very different size | n is large                          | Cluster densities differ |

## 3.5 COMPARISON

# Choosing an Algorithm With No Labels

## UNIT 3

**EXAM TIP** · A practical order of operations

1. **Look at the data first** plot two features, or a PCA projection. Shape is the single most informative thing you can observe.
2. **Scale unless you have a reason not to** all three methods measure distance, so an unscaled feature with a large range will decide the answer.
3. **Blobs and you can guess k** start with k-Means. It is fast, and cross-check k with silhouette.
4. **Odd shapes, or you want noise labelled** use DBSCAN, and choose  $\epsilon$  from a k-distance plot rather than by trial and error.
5. **You do not know k, and n is modest** use hierarchical clustering and inspect the dendrogram before cutting.
6. **Always compare against a baseline** run the same method on shuffled data. If your score is no better, you have found no structure.

**APPLY IT** GPS pings from delivery riders, and you want to find stop locations. Which algorithm, and why?

## UNIT III CONSOLIDATION

## UNIT 3

# Activity: Match the Algorithm to the Data

GROUP WORK · 12 minutes

Four datasets are described below. In groups of 3-4, choose an algorithm for each and defend it against the obvious alternative.

**1****The four datasets**

- Customer spend and visit frequency - roughly three round groups
- GPS pings along road networks - long and thin
- Sensor readings with 3% wild outliers
- Gene expression, 60 samples, 20,000 features

**2****For each, decide**

- Which algorithm, and which parameters
- Do you scale first? Why?
- How would you know if the answer were wrong?

**3****Argue the other side**

- Name the strongest alternative you rejected
- Say exactly what would go wrong if you used it

**4****The awkward one**

- Dataset 4 has far more features than samples
- What must you do before any clustering at all?

**DELIVERABLE** One speaker per group: 60 seconds on the dataset your group found hardest, and why.

## UNIT III RECAP

# Unit III - The Six Ideas Worth Remembering

## UNIT 3

**1****Hierarchical clustering builds every grouping once**

Agglomerative merges from  $n$  clusters down to one. You choose  $k$  afterwards by cutting the tree, with no re-run.

**2****Height in a dendrogram means dissimilarity**

A low join means two things are alike; a tall gap before the next merge suggests genuine structure and is a defensible place to cut.

**3****Linkage decides the shape of the clusters**

Single follows chains, complete makes compact balls, average compromises, Ward minimises the increase in within-cluster variance.

**4****Chaining is a weakness and a strength**

Single linkage welds groups through a thread of points - fatal on blobs, exactly right on crescents.

**5****DBSCAN grows clusters through dense regions**

A core point has at least  $\text{MinPts}$  neighbours within  $\epsilon$ . Border points sit on the edge; noise belongs to no cluster at all.

**6****Every algorithm fails where its assumption breaks**

k-Means assumes convex blobs, hierarchical assumes a meaningful merge criterion, DBSCAN assumes uniform density.

**NEXT UP** Unit IV - Dimensionality Reduction and Representation Learning: why high dimensions hurt, PCA's geometric intuition, manifold learning, t-SNE and UMAP, and the idea behind autoencoders.